SAVEETHA SCHOOL OF ENGINEERING, SIMATS

ECA14 - EMBEDDED SYSTEMS

CO1 - SYSTEM ARCHITECTURE-BASED ENGINEERING DESIGN ASSIGNMENT

ENGINEERING DESIGN REPORT

Networked Embedded Control System for a Luxury Automobile using CAN and CAN FD

REQUIREMENT > ALTERNATIVES > ANALYSIS > SELECTION > DESIGN > VERIFICATION > DEPLOYMENT DECISION

Item	Details
Student name	Natesa krishnan V
Register number	192511311
Department / Year	CSE/2nd year
Course outcome	CO1 - Analyze complex and networked embedded systems (CAN / CAN FD)
Bloom's levels	K4 Analyze | K5 Evaluate | K6 Design
Date of submission	02 November 2026

Summary of the Design

A luxury automobile needs engine control, braking, safety monitoring, body control and infotainment to cooperate in real time. This report follows the design process required by the assignment and records every engineering decision with its reason and a number that supports it.

Design outcome in one view

Processing (Decision 1): 32-bit lockstep automotive MCUs (ASIL-D) for the Engine, Braking, Safety, Body and Gateway ECUs; an application SoC with a safety MCU co-processor for Infotainment.

Network (Decision 2): hybrid, domain-based network joined by a central gateway: **Bus A** CAN FD 500 kbit/s / 2 Mbit/s (+ redundant channel A') for Engine, Braking, Safety; **Bus B** Classical CAN 125 kbit/s for Body; **Bus C** CAN FD 500 kbit/s / 2 Mbit/s for Infotainment.

Formal model (Decision 3): Mealy finite-state machine for the Emergency Braking function, with a Petri net for the concurrent activities.

Verification: Bus A load 9.9 % (Classical CAN would need 27.0 %); worst-case response of the emergency-brake message 0.52 ms against a 5 ms deadline; end-to-end latency 8.5 ms against 10 ms; Brake-ECU CPU utilisation 62.5 %.

Final decision: YES - deploy, subject to the HIL validation and ISO 26262 evidence listed in Section 7.

Contents

- Stage 1 - Interpreting the System Requirements
- Stage 2 - Analysing and Selecting the Embedded Architecture
- Stage 3 - Networked System Architecture
- Stage 4 - Analysing CAN and CAN FD
- Stage 5 - Formal Modelling of the Safety-Critical Function
- Stage 6 - Verification and Critical Evaluation
- Final Engineering Decision
- Appendix A - Python analysis code
- Appendix B - C implementation of the emergency-braking FSM
- Appendix C - Assumptions and references

Stage 1 - Interpreting the System Requirements

The first task is to turn the narrative problem into measurable requirements. Nothing in the problem statement fixes the ECU hardware, the protocol or the fault-tolerance scheme, so each requirement below is derived from the functions the car must perform and from the way they can fail.

1.1 Major functional requirements

Table 1.1 - Functional requirements

ID	Requirement	Owner	Timing
FR1	Engine control: fuel injection, ignition timing, throttle, idle & torque management	Engine ECU	Crank-synchronous, 1-10 ms loops
FR2	Braking: ABS, EBD, traction/stability intervention, brake-by-wire request handling	Braking ECU	5 ms control loop
FR3	Safety monitoring: collision detection, emergency braking request, airbag / pre-tensioner deployment, system health supervision	Safety ECU	5-20 ms; event latency < 10 ms
FR4	Body control: lighting, locks, windows, wipers, HVAC, immobiliser	Body-control ECU	20-100 ms
FR5	Infotainment: display, audio, navigation, telematics, smartphone integration	Infotainment ECU	20-50 ms, soft real-time
FR6	Cross-domain data exchange (vehicle speed, ignition state, warnings, diagnostics, OTA update) without compromising safety	Central gateway	50-100 ms

1.2 Role of the individual ECUs

Table 1.2 - ECU roles and criticality

ECU	Role	Class
Engine ECU	Closed-loop control of combustion and torque; publishes torque/RPM; obeys torque-limit requests from brake/stability control	Hard RT ASIL-C
Braking ECU	ABS/EBD/ESC control; reads wheel speeds and pressure; executes deceleration requests from the Safety ECU; commands hydraulic valves	Hard RT ASIL-D
Safety ECU	Fuses radar/camera/accelerometer data, computes time-to-collision (TTC), issues emergency-brake request (EBR), fires restraints, supervises the other ECUs by heartbeat	Hard RT ASIL-D
Body-control ECU	Low-speed comfort functions; aggregates many digital inputs/outputs, LIN sub-networks	Soft RT QM-ASIL-A
Infotainment ECU	High-throughput multimedia, navigation, connectivity; the only ECU exposed to external (USB/BT/Wi-Fi/cellular) interfaces	Soft RT QM
Central gateway	Routes, filters and rate-limits messages between domains; performs diagnostics/OTA; acts as security/safety firewall	Firm RT ASIL-B

1.3 Real-time communication requirements

A message is useful only if it arrives before its deadline. Hard real-time messages (braking, engine, safety) must never miss a deadline; soft real-time messages (body, infotainment) may be delayed or dropped occasionally.

Table 1.3 - Real-time message requirements

Message class	Rate	Latency requirement	Criticality
Emergency brake request (0x001)	Sporadic, min. inter-arrival 10 ms	<= 5 ms (bus) / <= 10 ms (end-to-end)	Hard
Wheel speed, brake status, engine torque, torque-limit	10 ms periodic	<= 10 ms	Hard
Engine data / radar object list	20 ms periodic	<= 20 ms	Hard / firm
Body status & commands	20-100 ms	= period	Soft
Infotainment / telematics	20-50 ms	= period (loss tolerated)	Soft
Diagnostics / software-update blocks	100 ms bursts	Best effort	None

1.4 Processing, memory and I/O requirements

The figures below are engineering estimates derived from typical production ECUs of each class (control-loop rate x instruction count per loop + operating-system overhead + 30 % margin). They size the processor choice in Stage 2.

Table 1.4 - Estimated processing, memory (Flash / RAM) and I/O

ECU	DMIPS	Flash / RAM	I/O	ASIL
Engine ECU	300-400	4 MB / 512 KB	8 analog in, crank/cam capture, 6 injector + 4 ignition drivers, throttle H-bridge	ASIL-C
Braking ECU	250	2 MB / 256 KB	4 wheel-speed inputs, pressure/yaw sensors, 12 valve drivers, pump motor	ASIL-D
Safety ECU	600	8 MB / 1 MB	Radar/camera interface, accelerometers, 16 squib drivers, 2 CAN FD	ASIL-D
Body-control ECU	80	1 MB / 128 KB	~40 digital in, ~30 outputs, 2 LIN, 1 CAN	QM / A
Central gateway	200	2 MB / 512 KB	3-4 CAN FD controllers, Ethernet (future)	ASIL-B
Infotainment ECU	10 000+	2 GB DRAM / 8 GB eMMC	LVDS display, audio I2S, USB, BT/Wi-Fi, GPS, CAN FD	QM

1.5 Reliability and fault-tolerance requirements

Table 1.5 - Reliability requirements

Aspect	Requirement
Fault detection	Lockstep core comparison, ECC on Flash/RAM, window watchdog, power-on and run-time self-test, plausibility checks

Aspect	Requirement
Communication integrity	CAN CRC (HD = 6) + application-level E2E protection (alive counter + CRC-8), timeout supervision of every safety message
Fail-safe / fail-operational	Defined safe state per ECU; hydraulic (mechanical) braking always available; redundant CAN FD channel for safety messages
Safety integrity	ISO 26262 targets: braking and safety ECUs ASIL-D (PMHF < 10 FIT), engine ASIL-C, body ASIL-A/QM
Availability	Single-fault tolerance on the safety bus; graceful degradation; bus-off recovery

1.6 Communication between safety-critical and non-safety-critical ECUs

- The Infotainment and Body ECUs may read limited vehicle state (speed, gear, warnings) but must never be able to write to the safety bus.
- Safety-critical ECUs may send status to non-critical ECUs (one-way, filtered, rate-limited) for display purposes.
- Any traffic crossing domains passes a gateway that applies an ID whitelist, rate limiting and authentication (freedom from interference, ISO 26262-6).
- Non-critical traffic must not increase the worst-case latency of safety messages; hence they are placed on separate physical buses.

1.7 Constraints affecting processor and protocol selection

Table 1.6 - Constraints and their consequences

Constraint	Consequence for design
Temperature -40 to +125 C, AEC-Q100 Grade 1, 12 V supply with load-dump / cranking dips	Only automotive-qualified MCUs/SoCs; supply monitors needed
Hard real-time deadlines (ms) and determinism	Rules out general-purpose OS on MPU for control loops; favours MCU + RTOS (AUTOSAR Classic)
Functional safety (ISO 26262) up to ASIL-D	Needs lockstep, ECC, safety manual, certified toolchain
Wiring cost, weight, EMC	Shared bus (CAN/CAN FD) instead of point-to-point; twisted pair; common-mode choke
Legacy ecosystem and cost	Classical CAN well-proven for low-rate body functions; CAN FD needs FD-capable controllers & transceivers
Future data growth (ADAS, OTA)	Bandwidth headroom: CAN FD with 2-5 Mbit/s data phase; Ethernet backbone as migration path
Security (external connectivity)	Gateway firewall, authenticated diagnostics, rate limiting

Stage 1 Deliverable - System Requirement Statement

The proposed embedded system shall provide six cooperating automotive functions (engine, braking, safety, body, infotainment, gateway) on dedicated ECUs connected by a networked embedded architecture such that: (a) all hard real-time messages are delivered within their deadlines (emergency-brake request <= 5 ms on the bus, <= 10 ms end-to-end from collision detection to brake command) under worst-case arbitration and one transmission error; (b) the safety-critical domain meets ISO 26262 ASIL-D by detecting and containing single faults, with a defined safe state and mechanical braking fallback; (c) non-critical ECUs cannot interfere with safety-critical communication; (d) bus load stays below 70 % with at least 2x growth in signals, and the network supports payloads larger than 8 bytes and data rates beyond 1 Mbit/s; and (e) operation is guaranteed from -40 to +125 C with 12 V supply variations at automotive cost.

Stage 2 - Analysing and Selecting the Embedded Architecture

2.1 Alternatives

Four processing architectures are considered: MCU (CPU, Flash, RAM and peripherals on one chip), MPU (CPU with external memory and an MMU-based operating system), DSP (architecture optimised for multiply-accumulate and streaming signal processing) and SoC (multiple cores, GPU/accelerators, memory controllers and interfaces integrated).

Table 2.1 - Comparison of processing architectures

Criterion	MCU	MPU	DSP	SoC
Processing capability	50-600 DMIPS (up to ~1 GHz multicore in high-end)	1-10 k DMIPS, MMU, Linux/QNX	Very high MAC/s for filters, FFT, radar	10 k-100 k+ DMIPS, + GPU/NPU/DSP
Memory	On-chip Flash 1-16 MB, SRAM 0.1-4 MB; small, deterministic	External DRAM/Flash (GBs); non-deterministic latency	On-chip scratchpad; external as needed	Large external DRAM + on-chip caches
Power	0.2-2 W	2-8 W	0.5-3 W	5-20 W
Real-time response	Excellent: interrupt latency ~ 1 us, no cache/MMU jitter	Poor-moderate: OS, caches, MMU	Excellent for streaming; weak for control logic	Moderate; needs hypervisor / R5 cores for RT
Integration	CAN FD, ADC, PWM, timers, safety logic on one chip	Needs external PMIC, Flash, many peripherals	Needs host MCU for I/O and CAN	Highest: CPU+GPU+codecs+CAN FD+Ethernet
Cost / complexity	Low cost (USD 5-20), simple RTOS	Medium (USD 20-60), complex software stack	Medium, specialised toolchain	High (USD 40-150), complex BSP and validation
Automotive suitability	Industry standard for ECUs (ASIL-D lockstep parts exist)	Used for ADAS/IVI only	Radar/audio accelerators inside MCU/SoC	Standard for IVI, ADAS domain controllers

2.2 Quantitative selection (weighted decision matrix)

Each architecture is scored 1 (poor) to 5 (excellent) per criterion; criteria are weighted by importance to the application (real-time response and automotive suitability weigh most because a missed deadline or an unqualified part is unacceptable). Weighted score = sum(weight x score) / 100.

$$\text{Score} = \text{Sum}(w_i \times s_i) / 100$$

Table 2.2 - Decision matrix for the control ECUs (Engine, Braking, Safety, Body, Gateway)

Criterion	Weight	MCU	MPU	DSP	SoC
Processing capability	15	3	4	4	5
Memory	10	3	5	3	5
Power	10	5	2	3	2
Real-time response	20	5	2	4	3
Integration	10	4	3	2	5
Cost / complexity	15	5	3	3	2
Automotive suitability	20	5	2	3	3
Weighted score (max 5.00)	**100**	**4.40**	**2.85**	**3.25**	**3.45**

Worked example (MCU): (15x3 + 10x3 + 10x5 + 20x5 + 10x4 + 15x5 + 20x5)/100 = (45+30+50+100+40+75+100)/100 = **4.40**.

Table 2.3 - Decision matrix for the Infotainment ECU

Criterion	Weight	MCU	MPU	DSP	SoC
Processing capability	15	1	4	3	5
Memory	10	1	4	2	5
Power	10	5	3	3	3
Real-time response	20	4	2	3	3
Integration	10	2	3	2	5
Cost / complexity	15	5	3	3	3
Automotive suitability	20	4	3	2	4
Weighted score (max 5.00)	**100**	**3.30**	**3.05**	**2.60**	**3.90**

For control ECUs the MCU wins by a wide margin (4.40 vs 3.45 for the next best). For infotainment, the SoC wins (3.90) because the workload (graphics, audio, navigation, connectivity) needs 10 000+ DMIPS and gigabytes of memory, which no MCU can provide. A DSP is not chosen as a stand-alone ECU processor because the system has no dominant signal-processing workload; radar pre-processing, where a DSP is valuable, is available as an on-chip accelerator in modern safety MCUs.

Design Decision 1

I recommend a heterogeneous, MCU-centred architecture - a 32-bit lockstep dual-core automotive MCU (ASIL-D class, 200 MHz class, with ECC Flash/RAM, CAN FD controllers and hardware safety monitors) for the Engine, Braking, Safety, Body-control and Gateway ECUs, and an application SoC with an MCU safety co-processor for the Infotainment ECU, because (i) the control loops need deterministic, microsecond-latency interrupt response, which MCUs provide and MPUs/SoCs cannot guarantee; (ii) the Brake ECU needs only ~250 DMIPS and its task set uses 62.5 % of a 200 MHz core (Section 6.5), so a more powerful processor would waste power and cost; (iii) automotive-qualified MCUs integrate CAN FD, ADC, PWM and lockstep safety features on one chip, minimising component count and certification effort; and (iv) infotainment is the only function whose compute and memory needs (10 000+ DMIPS, GB-class DRAM) justify a higher-power, higher-cost SoC, and isolating it there keeps its complexity out of the safety domain.

Stage 3 - Developing the Networked System Architecture

3.1 System architecture diagram

The architecture is **domain-based**: three bus segments grouped by criticality and data rate, connected through a central gateway. Colours in Figure 3.1 identify the domains: red - safety-critical/powertrain, blue - body, green - infotainment.

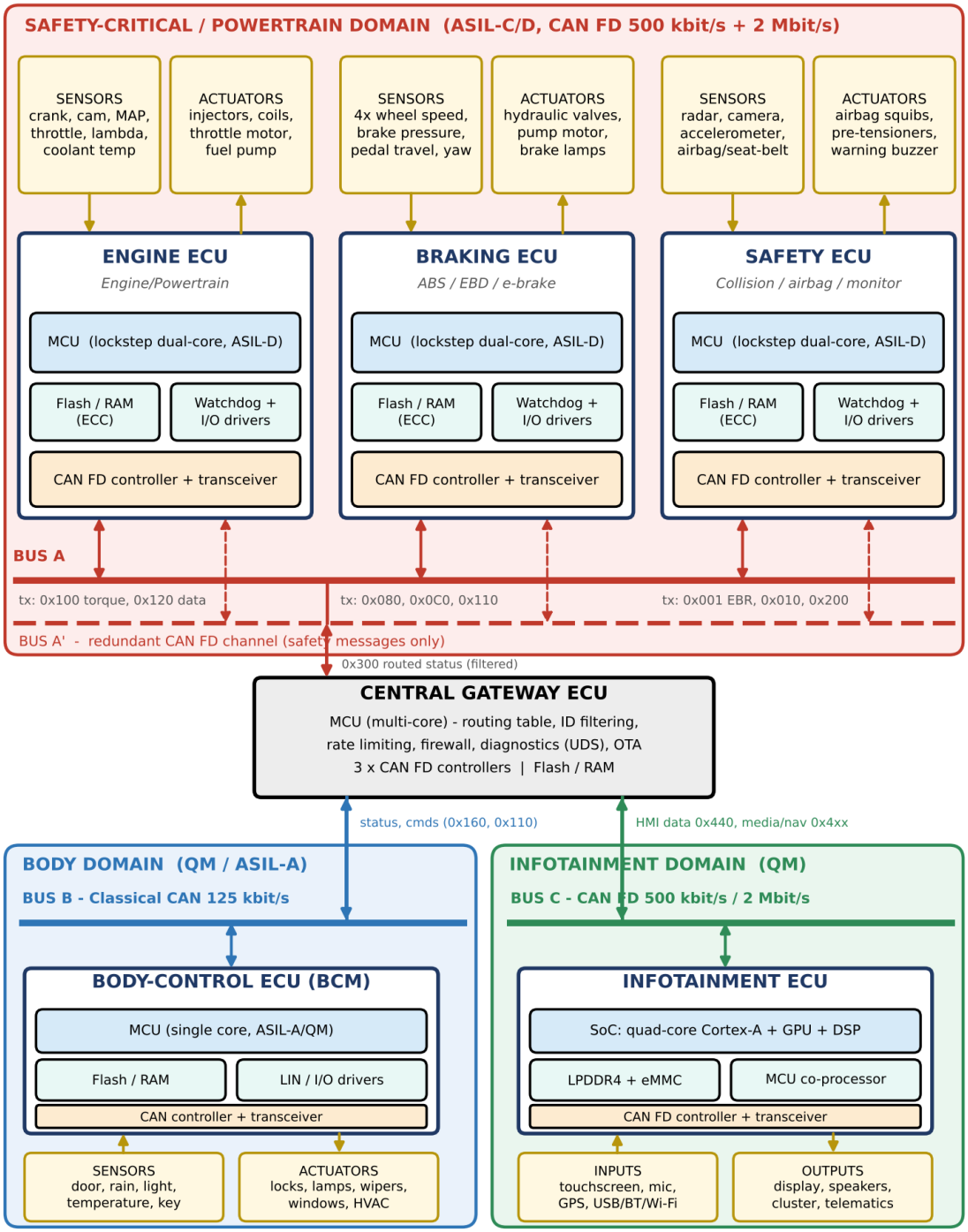


Figure 3.1 - Proposed networked embedded-system architecture (ECUs, sensors, actuators, processing units, memory, CAN/CAN FD buses and gateway)

3.2 Function of each ECU

Table 3.1 - ECU functions and bus connection

ECU	Function	Bus
Engine ECU	Reads crank/cam/MAP/throttle/lambda/temperature; computes injection & ignition; drives injectors, coils, throttle motor; reports torque/RPM; accepts torque-limit.	Bus A, A'
Braking ECU	Reads 4 wheel speeds, pressure, yaw; runs ABS/EBD/ESC; executes deceleration requests; drives valves & pump; reports wheel speed and status.	Bus A, A'
Safety ECU	Fuses radar/camera/accelerometer; computes TTC; sends EBR; fires airbags; monitors heartbeats of Engine/Brake ECUs; commands safe state.	Bus A, A'
Body-control ECU	Lighting, locks, windows, wipers, HVAC, immobiliser; low-rate sensors and actuators; local LIN nodes.	Bus B
Infotainment ECU	Media, navigation, telematics, HMI; receives vehicle info for display; sends control requests.	Bus C
Central gateway	Whitelist routing, protocol translation (CAN FD <-> Classical CAN), rate limiting, diagnostics (UDS), OTA staging.	A, B, C

3.3 Information exchanged and direction of data flow

Table 3.2 - Bus A message matrix (CAN FD 500 kbit/s / 2 Mbit/s)

CAN ID	Message	Direction	Content	Period	Bytes
0x001	EmergencyBrakeRequest	Safety -> Brake (all)	Decel request, EBR flag, alive counter, CRC	10 ms MIT, sporadic	8
0x010	SafetyHeartbeat	Safety -> all	Health, mode, alive counter	10 ms	8
0x080	WheelSpeed	Brake -> Engine, Safety	4 wheel speeds, vehicle speed	10 ms	8
0x0C0	BrakeStatus	Brake -> Safety, Gateway	Pressure, ABS active, pedal	10 ms	8
0x100	EngineTorque	Engine -> Brake, Safety	Actual/requested torque, RPM	10 ms	8
0x110	TorqueLimitReq	Brake -> Engine	Torque cut/limit (ABS/TCS)	10 ms	8
0x120	EngineData	Engine -> Gateway	Temperatures, fuel, diagnostics	20 ms	16
0x200	RadarObjectList	Safety -> Brake	Tracked objects (range, velocity)	20 ms	32
0x300	GatewayRouted	Gateway -> Bus A	Ignition, gear, filtered body status	50 ms	8

CAN ID	Message	Direction	Content	Period	Bytes
0x7E0	DiagSoftwareBlock	Gateway <-> any	UDS diagnostics / OTA block	100 ms	64

Data flow follows the control chain sensor -> ECU -> bus -> ECU -> actuator:

1. Radar/camera/accelerometer -> Safety ECU computes TTC -> 0x001 EBR over Bus A and A' -> Braking ECU -> valves/pump (actuators).
2. Wheel-speed sensors -> Braking ECU -> 0x080 -> Engine ECU (traction torque cut) and Safety ECU (vehicle speed for TTC).
3. Engine ECU -> 0x100/0x120 -> Braking and Safety ECUs (torque, RPM) and Gateway (diagnostics).
4. Gateway -> 0x300 (ignition, gear) -> Bus A (read-only for safety ECUs); Gateway <-> Bus B/C: status and commands in both directions, filtered by the routing table.

3.4 Role of the communication bus

The CAN/CAN FD bus is a shared, multi-master, broadcast medium: every ECU sees every frame and decides by identifier whether to accept it. Its roles are (1) transport - replaces point-to-point wiring (one twisted pair per domain instead of dozens of dedicated wires); (2) prioritisation - the identifier doubles as the priority, so urgent frames win access without a central scheduler (Section 4.2); (3) fault detection and containment - CRC, acknowledgement and error counters isolate faulty nodes; and (4) synchronisation - all nodes resynchronise on every frame edge, giving a common time base for sampling.

3.5 Separation of safety-critical and non-critical functions

- **Physical separation:** Engine, Braking and Safety ECUs share Bus A/A' only; Body is on Bus B; Infotainment is on Bus C. A babbling infotainment node can never occupy Bus A.
- **Gateway firewall:** only whitelisted IDs cross domains, with per-ID rate limits; infotainment cannot send any Bus A ID, and diagnostic/OTA access requires authentication.
- **Processor separation:** safety functions run on lockstep ASIL-D MCUs; the SoC in the infotainment unit runs a rich OS (Linux/Android Automotive) that is never trusted for safety functions.
- **Priority separation:** safety IDs occupy the lowest identifier range (0x001-0x2FF); diagnostic and gateway traffic use higher numbers and therefore always lose arbitration.
- **Redundancy only where needed:** the second channel A' exists only in the safety domain, where a single-fault tolerance requirement exists.

Stage 4 - Analysing CAN and CAN FD

4.1 Frame-level comparison

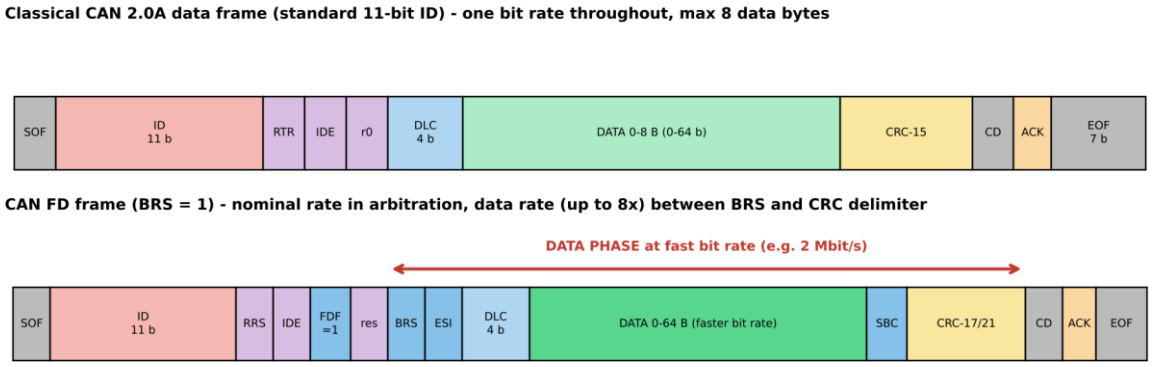


Figure 4.1 - Classical CAN and CAN FD frame formats (fields drawn approximately to scale)

Table 4.1 - Classical CAN vs CAN FD

Parameter	Classical CAN (ISO 11898-1)	CAN FD
Frame structure	SOF, 11/29-bit ID, RTR, IDE, r0, DLC, Data, CRC-15, ACK, EOF	Adds FDF, BRS, ESI flags; stuff-count field; CRC-17/21; fixed stuff bits in CRC
Identifier format	11-bit (base) or 29-bit (extended)	Same 11/29-bit; no RTR frames
Payload size	0-8 bytes	0-64 bytes (DLC 9-15 = 12, 16, 20, 24, 32, 48, 64 B)
Data rate	<= 1 Mbit/s	Nominal <= 1 Mbit/s in arbitration; data phase 2-5 Mbit/s typ. (up to 8 Mbit/s)
Arbitration	Bit-wise, non-destructive, lowest ID wins	Identical; arbitration always at nominal rate
Synchronization	Hard sync at SOF, resync on edges; single bit timing (sample point ~ 80 %)	Two bit timings; switch at BRS and back at CRC delimiter; transceiver delay compensation (SSP) needed
Error handling	Bit, stuff, form, CRC, ACK errors; TEC/REC counters; HD = 6	Same mechanisms; stronger CRC (HD = 6 for 64 B); ESI flag; stricter bit-timing demands
Real-time behaviour	Deterministic, ~270 us for 8 B at 500 kbit/s; long payloads need segmentation (ISO-TP)	~120 us for 8 B, ~403 us for 64 B (500k/2M): lower latency & load, no segmentation
Automotive use	Legacy powertrain/body, mature, cheap	New powertrain/chassis/ADAS, gateways, flashing; supported by AUTOSAR 4.x

4.2 Situation 1 - simultaneous transmission with a safety-critical message

Assume bus idle after the inter-frame space. The Engine ECU (ID 0x120), the Braking ECU (ID 0x110) and the Safety ECU (ID 0x001, emergency brake request) all start transmitting at the same bit time. All nodes send SOF (dominant, 0) together and then their 11-bit ID, most-significant bit

first. The bus is wired-AND: a dominant 0 overrides a recessive 1. A node that sends 1 but reads 0 has lost and immediately becomes a receiver, **without destroying the winner's frame**.

Table 4.2 - Identifier comparison (bit ID10 ... ID0)

ECU	ID	Binary (ID10..ID0)	Outcome
Safety ECU	0x001	000 0000 0001	Sends 0 at ID8 where others send 1 -> **WINS**
Braking ECU	0x110	001 0001 0000	Sends 1 at ID8, reads 0 -> loses at ID8
Engine ECU	0x120	001 0010 0000	Sends 1 at ID8, reads 0 -> loses at ID8

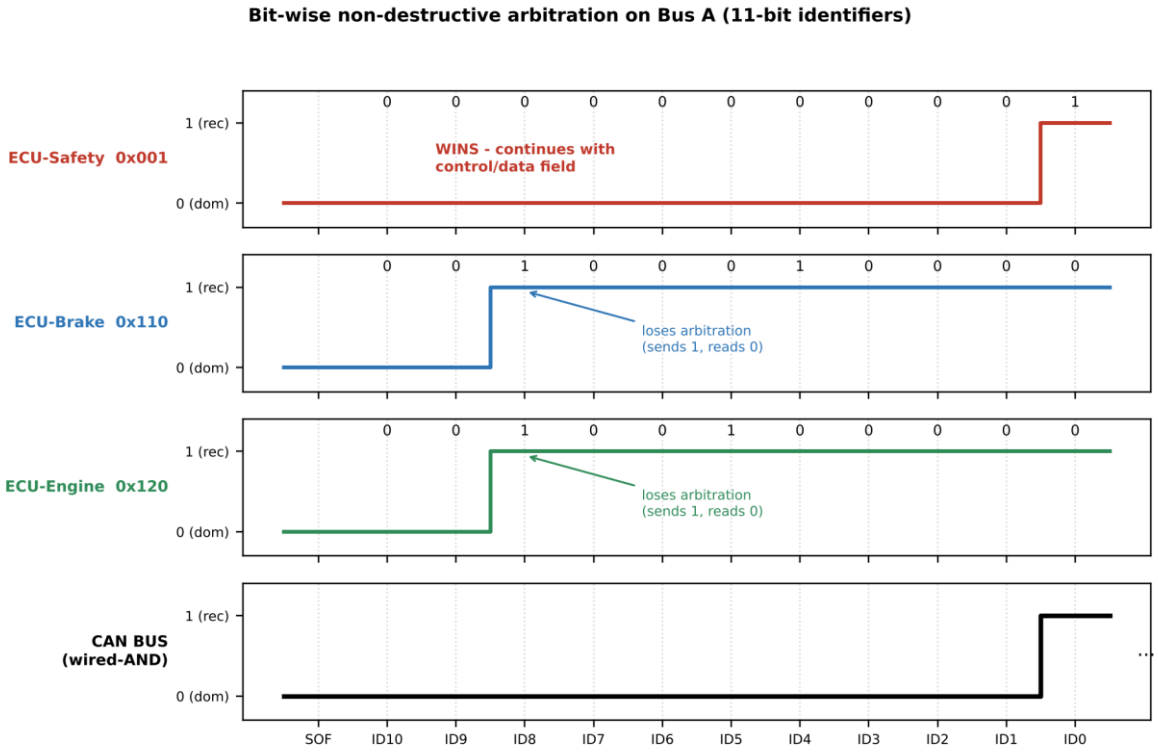


Figure 4.2 - Arbitration waveforms: the lowest identifier (0x001) wins at bit ID8

Result: the Safety ECU's frame is transmitted at once; 0x110 and 0x120 automatically retry at the next idle period. In that second arbitration 0x110 and 0x120 first differ at bit ID5 (0x110 sends 0, 0x120 sends 1), so 0x110 wins and 0x120 goes last. Using the CAN FD frame time of 120.5 us for an 8-byte payload (Section 4.4):

Safety frame ends at 120.5 us; Braking frame at 241.0 us; Engine frame at 361.5 us

The lowest-priority frame therefore waits only 0.36 ms - far inside its 10 ms deadline - and no bandwidth is wasted by collisions (unlike Ethernet CSMA/CD). In the worst case the Safety frame is blocked only by a frame already on the bus (arbitration is non-preemptive): at most 403 us (one 64-byte diagnostic frame), which is why Bus A has a worst-case response of 523.5 us for 0x001 (Section 6.3).

4.3 Situation 2 - payload larger than Classical CAN allows

The Safety ECU must send a radar object list of **32 bytes every 20 ms**, and the Gateway a 64-byte diagnostic/OTA block. A Classical CAN frame carries at most 8 bytes, so the data must be segmented.

Table 4.3 - Time and efficiency to move one application payload (500 kbit/s nominal, 2 Mbit/s data phase)

Payload	Classical CAN frames	Classical total time	CAN FD frame time	Speed-up	Effective throughput (Cl. / FD)
8 B	1	270 us	120.5 us	2.2x	237 / 531 kbit/s
16 B	2	540 us	160.5 us	3.4x	237 / 798 kbit/s
32 B	4	1080 us	243.0 us	4.4x	237 / 1053 kbit/s
64 B	8	2160 us	403.0 us	5.4x	237 / 1270 kbit/s

Calculation for the 32 B radar list: Classical CAN needs ceil(32/8) = 4 frames x 270 us = 1080 us (5 frames x 270 = 1350 us if ISO-TP's 1-byte header is used, 7 B per frame); CAN FD needs one frame of 243.0 us - 4.4 times faster. Segmentation in Classical CAN also lets other messages interleave between segments (the receiver must reassemble, and a lost segment loses the whole object list), while CAN FD delivers it atomically with a stronger CRC. CAN FD should therefore be introduced for this traffic. At 8 B, FD already saves 55 % of bus time (120.5 vs 270 us) because the data bytes are sent at 4x the bit rate.

4.4 Frame-time calculations

Classical CAN, standard frame, worst case with bit stuffing (3-bit inter-frame space included):

$$C_cl(n) = (8n + 47 + \text{floor}((34 + 8n - 1)/4)) \times t_bit \rightarrow n = 8: 64 + 47 + 24 = 135 \text{ bits} = 270 \text{ us at } 500 \text{ kbit/s}$$

CAN FD (11-bit ID, BRS = 1), n = 8 bytes, 500 kbit/s nominal and 2 Mbit/s data. Bits at the nominal rate: SOF+ID+RRS+IDE+FDF+res (16) + BRS (1) + stuff (3) + ACK, ACK-delimiter, EOF, IFS (12) = 32 bits = 64.0 us. Bits at the data rate: ESI+DLC+data = 69, dynamic stuff floor(68/4) = 17, stuff count 4, CRC-17 + 5 fixed stuff = 22, CRC delimiter 1 = 113 bits = 56.5 us.

$$C_FD(8) = 64.0 \text{ us} + 56.5 \text{ us} = 120.5 \text{ us} \quad (64 \text{ B: } 32 \text{ nominal bits} + 678 \text{ data bits} \rightarrow 64 + 339 = 403 \text{ us})$$

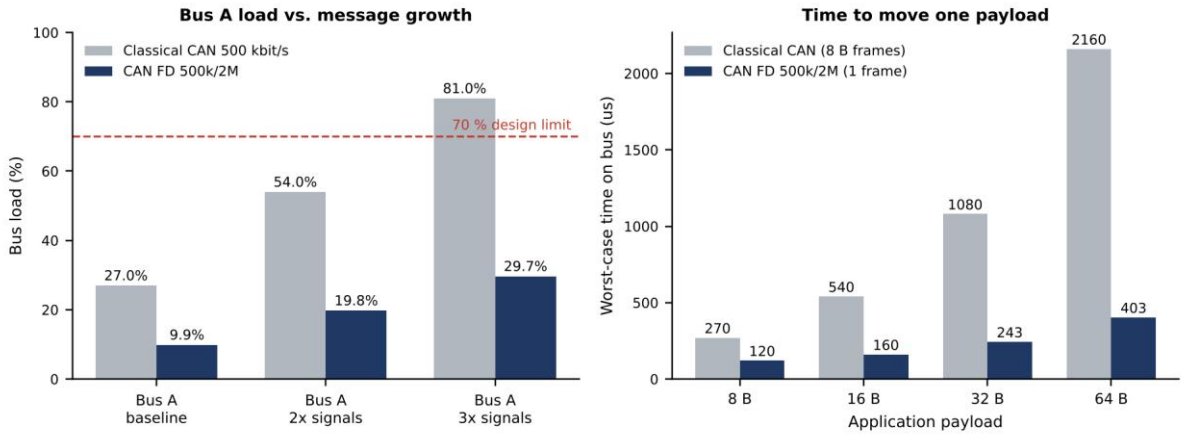


Figure 4.3 - Left: Bus A load for Classical CAN vs CAN FD as signals grow; right: time to transfer one payload

4.5 Decision

Design Decision 2

I recommend a hybrid CAN / CAN FD architecture: CAN FD (500 kbit/s arbitration, 2 Mbit/s data phase) for Bus A (Engine, Braking, Safety, with redundant channel A') and for Bus C (Infotainment), and Classical CAN at 125 kbit/s for Bus B (Body-control), with a central gateway translating between them, because Bus A carries the hard real-time and larger payloads (16-64 B radar, engine data, diagnostics) - CAN FD lowers its load from 27.0 % to 9.9 % and the worst-case latency of the emergency message, and keeps 2x-3x growth headroom (19.8 % / 29.7 % vs 54 % / 81 % for Classical CAN); the CRC-17/21 and room for alive counter + CRC-8 in the same frame strengthen E2E protection; Infotainment traffic (16-64 B) cuts from 22.1 % to 4.8 %; Body functions are slow (20-100 ms, 8 B), cheap Classical CAN transceivers suffice and its load is only 14.0 % at 125 kbit/s, so upgrading would add cost with no benefit.

Table 4.4 - Protocol assignment and bit-timing configuration

Bus	Nodes	Protocol / rate	Bit timing (80 MHz CAN clock)	Sample point
A / A'	Engine, Braking, Safety, Gateway	CAN FD 500 kbit/s / 2 Mbit/s	Nominal: BRP 10, 16 tq (Seg1 12, Seg2 3, SJW 3) Data: BRP 2, 20 tq (Seg1 15, Seg2 4, SJW 4), TDC on	81.25 % / 80 %
B	BCM, Gateway	Classical CAN 125 kbit/s	BRP 40, 16 tq (Seg1 12, Seg2 3, SJW 3)	81.25 %
C	Infotainment, Gateway	CAN FD 500 kbit/s / 2 Mbit/s	As Bus A	81.25 % / 80 %

A 5 Mbit/s data phase is possible (an 8 B frame then takes 86.6 us) but requires signal-integrity transceivers (SIC) and a controlled star/line topology; 2 Mbit/s is chosen as a robust first release with an upgrade path.

Stage 5 - Formal Modelling of the Embedded System

5.1 Selected function: Autonomous Emergency Braking (AEB)

The function is hosted on the Safety ECU and requests deceleration from the Braking ECU. It is safety-critical (ASIL-D), event-driven, and has clearly separated modes - an excellent candidate for a state machine. The model is a Mealy FSM: outputs depend on the current state and the input event, so reaction to an input (for example sending the brake request) occurs on the transition, without an extra state or delay.

5.2 States, inputs and outputs

Table 5.1 - States

State	Meaning
S0 INIT	Power-on self-test of MCU, memory, sensors
S1 STANDBY	System healthy; vehicle stopped or slow (v <= 8 km/h) - function inactive
S2 MONITOR	Armed; continuously computes time-to-collision (TTC)
S3 WARNING	TTC < 2.6 s; driver warned (visual + audible)
S4 PRE-BRAKE	TTC < 1.8 s; hydraulic pre-fill, light braking 2 m/s2
S5 EMERGENCY BRAKE	TTC < 1.0 s and driver not braking; full braking 9 m/s2, engine torque cut
S6 HOLD	Vehicle stopped; keep brakes applied for 1 s (200 ticks of 5 ms)
S7 DEGRADED	Brake ECU heartbeat lost; warn, driver braking only
S8 SAFE	Fault detected (lockstep/E2E/sensor); function disabled and latched until restart

Table 5.2 - Inputs (events) and outputs (actions)

Inputs / events	Outputs / actions
self_test_ok, fault, ignition_on	warn_hmi - buzzer + cluster warning
TTC (time-to-collision, from sensor fusion)	prefill - hydraulic pre-fill of brake circuit
speed v (km/h)	decel_req (m/s2) - value in CAN message 0x001
driver_brake, driver_override (steering/accelerator)	limit_torque - request to Engine ECU (0x110)
bus_alive (Brake ECU heartbeat)	(tick: 5 ms task of the Safety ECU)

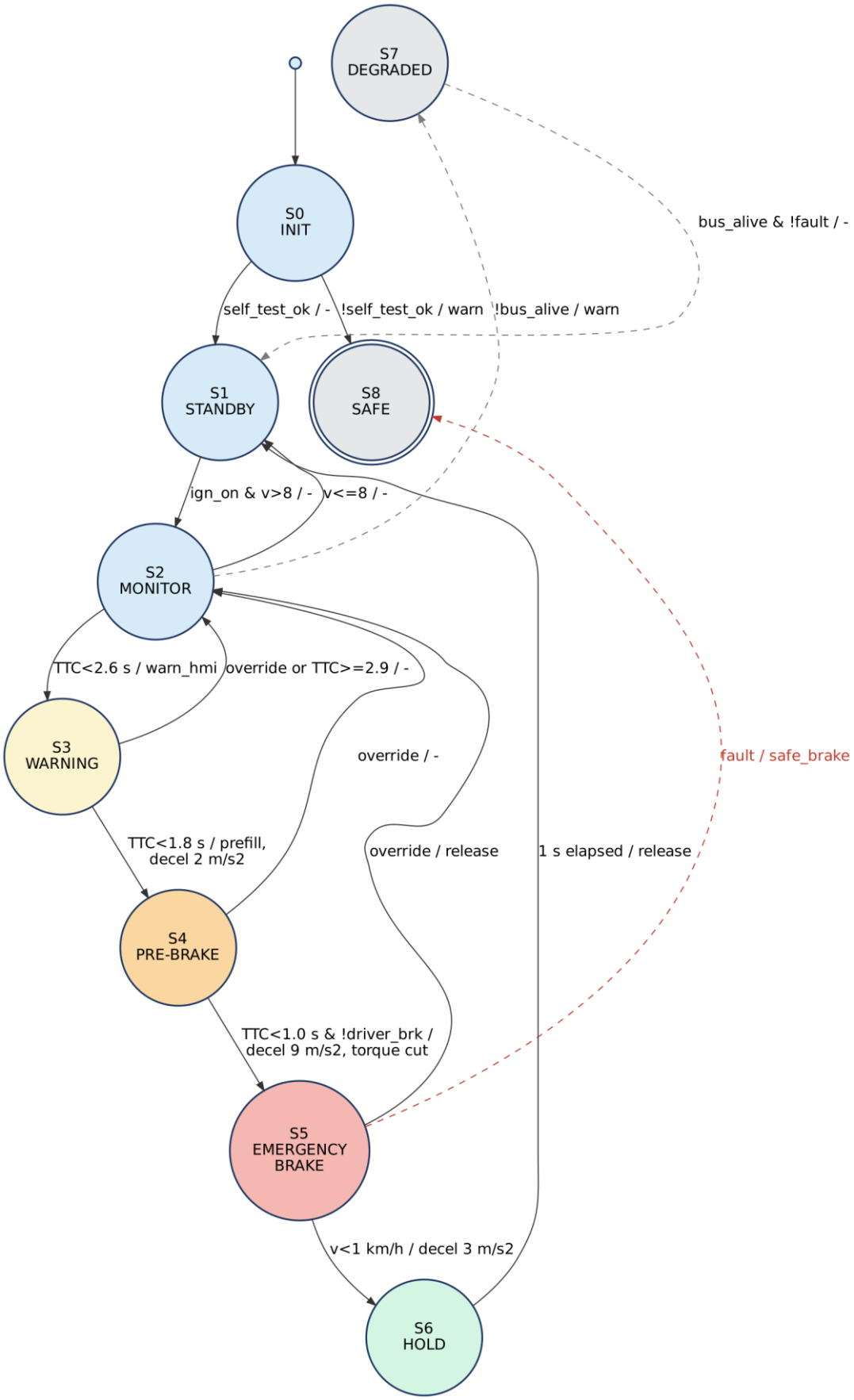


Figure 5.1 - Mealy FSM of the emergency-braking function (transition label = event / output; dashed = fault and degraded paths)

Table 5.3 - Transition table (Mealy: next state / output)

From	Event (guard)	To	Output
S0	self_test_ok	S1	-
S0	!self_test_ok	S8	warn_hmi
S1	ignition_on and v > 8	S2	-
S2	v <= 8	S1	-
S2	TTC < 2.6 s	S3	warn_hmi
S3	override or TTC >= 2.9 s	S2	-
S3	TTC < 1.8 s	S4	prefill, decel 2 m/s2
S4	TTC < 1.0 s and !driver_brake	S5	decel 9 m/s2, limit_torque
S4	override	S2	-
S5	v < 1 km/h	S6	decel 3 m/s2
S5	override	S2	release
S6	1 s elapsed	S1	release
any (S1-S6)	fault	S8	none (safe)
any (S1-S6)	!bus_alive	S7	warn_hmi
S7	bus_alive and !fault	S1	-

5.3 Concurrent activities - Petri net

The FSM captures the control logic but not the concurrent activities that make up one braking event: sensor acquisition, TTC computation, bus transmission, actuation and the watchdog all run in parallel. A Petri net models them: places (circles) hold tokens that represent conditions; transitions (boxes) fire when all their input places hold a token. T1 needs both a radar frame (P1) and a wheel-speed sample (P2) - synchronisation; T4/T5 form the watchdog loop that runs concurrently with the braking chain; the inhibitor arc from P8 prevents T3 (apply braking) once the watchdog requested the safe state.

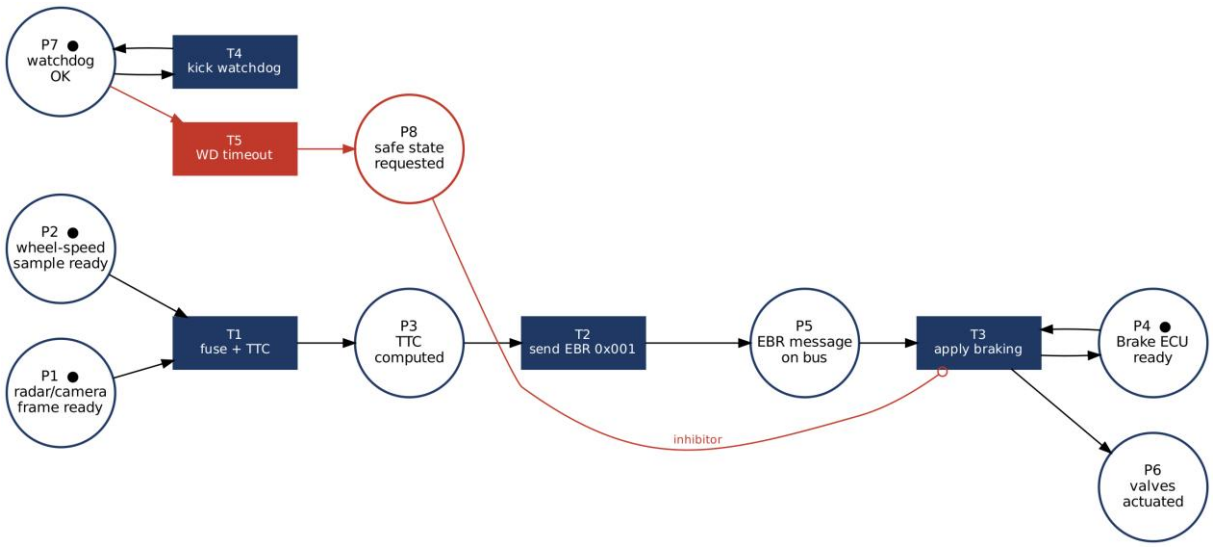


Figure 5.2 - Petri net of concurrent activities (initial marking: P1, P2, P4, P7)

Reachability check from the initial marking M0 = {P1, P2, P4, P7}: T1 fires -> P3; T2 -> P5; T3 (needs P5 and P4, P8 empty) -> P6 and returns the token to P4 (Brake ECU ready again); the watchdog loop T4 keeps P7 marked. If T5 fires (P7 -> P8) then T3 is inhibited, so no braking command can be issued from an unsupervised Safety ECU. The net is bounded (1-safe) - no place ever holds more than one token - so there is no buffer overflow, and the braking chain is deadlock-free because every transition has its input tokens produced by the preceding transition.

5.4 Verification of the model by execution

The FSM was implemented in C (Appendix B) and run against a scripted scenario: the car closes on an obstacle (TTC falls 9.9 -> 0.7 s) at 50 km/h and then a fault is injected. Output of the self-checking test:

```
tick  TTC   state      warn pre  decel torque
  0   9.9  MONITOR     0    0    0.0   0
  1   9.9  MONITOR     0    0    0.0   0
  2   2.5  WARNING     1    0    0.0   0
  3   2.0  WARNING     1    0    0.0   0
  4   1.7  PREBRAKE    1    1    2.0   0
  5   0.9  EMERGENCY   1    1    9.0   1
  6   0.8  EMERGENCY   1    1    9.0   1
  7   0.7  HOLD        1    1    9.0   1
fault injected -> SAFE
TEST PASSED
```

The sequence MONITOR -> WARNING -> PRE-BRAKE -> EMERGENCY (-> HOLD when the car has stopped) occurs at exactly the TTC thresholds of Table 5.3, and the injected fault drives the machine to SAFE, confirming the model.

Design Decision 3

I chose a Mealy finite-state machine (complemented by a Petri net for concurrency) to model the emergency-braking function because its behaviour is a finite set of modes (idle, monitor, warn, pre-brake, brake, hold, degraded, safe) with transitions triggered by discrete events and thresholds; a Mealy machine lets outputs (the CAN brake request, the warning) be produced in the same 5 ms tick as the event, giving the lowest reaction delay and fewer states than a Moore machine would need; the transition table maps directly to a switch-case C implementation that can be exhaustively tested and checked for unreachable states or missing transitions (important for ISO 26262); and the Petri net adds what an FSM cannot express - parallel sensing, communication, actuation and watchdog activity, synchronisation and the inhibit rule. UML/SysML would suit documentation, but gives no direct, analysable semantics for this small controller.

Stage 6 - Verification and Critical Evaluation

The architecture is verified quantitatively with the Python model in Appendix A (frame times, bus load, worst-case response times) and by schedulability analysis of the Brake ECU. Message sets are those of Table 3.2 (Bus A) and the Body and Infotainment sets of Appendix A.

6.1 Real-time performance and bus load

$$\text{Bus load } U = \sum_i \left(C_i / T_i \right)$$

Table 6.1 - Bus-load results

Network	Protocol	Frames	Load	Design limit
Bus A	Classical CAN 500 kbit/s (comparison)	21	27.0 %	70 %
Bus A	**CAN FD 500k / 2M (chosen)**	10	**9.9 %**	70 %
Bus B	Classical CAN 125 kbit/s	7	14.0 %	70 %
Bus C	CAN FD 500k / 2M	5	4.8 %	70 %
Bus C	Classical CAN 500 kbit/s (comparison)	-	22.1 %	70 %

6.2 CAN arbitration and worst-case response time

Because CAN is non-preemptive and priority-based, the worst-case response time of message i is (Tindell/Davis analysis):

$$R_i = w_i + C_i , \quad w_i = B_i + \sum_{j \in hp(i)} \text{ceil}\left((w_i + t_{bit}) / T_j \right) \times C_j \quad (+ E(w_i) \text{ for errors})$$

where B_i is the blocking by one lower-priority frame already on the bus (max 403 us here) and E the error-recovery overhead (31 bit-time error frame + retransmission of the largest affected frame). The recurrence is solved iteratively.

Table 6.2 - Worst-case response times on Bus A (CAN FD) vs deadlines

ID	Message	C (us)	B (us)	R fault-free (us)	R with 1 error (us)	Deadline (us)	Met
0x001	EmergencyBrakeRequest	120.5	403.0	523.5	706.0	5000	**YES**
0x010	SafetyHeartbeat	120.5	403.0	644.0	826.5	10000	**YES**
0x080	WheelSpeed	120.5	403.0	764.5	947.0	10000	**YES**
0x0C0	BrakeStatus	120.5	403.0	885.0	1067.5	10000	**YES**
0x100	EngineTorque	120.5	403.0	1005.5	1188.0	10000	**YES**
0x110	TorqueLimitReq	120.5	403.0	1126.0	1308.5	10000	**YES**
0x120	EngineData	160.5	403.0	1286.5	1509.0	20000	**YES**
0x200	RadarObjectList	243.0	403.0	1529.5	1834.5	20000	**YES**
0x300	GatewayRouted	120.5	403.0	1650.0	1955.0	50000	**YES**
0x7E0	DiagSoftwareBlock	403.0	0.0	1650.0	2115.0	100000	**YES**

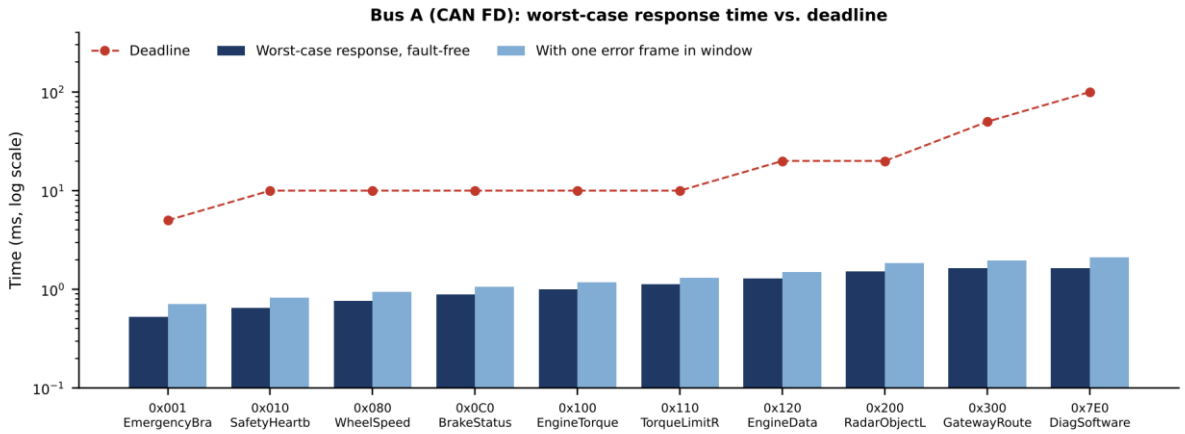


Figure 6.1 - Worst-case response time of each Bus A message compared with its deadline (log scale)

The emergency-brake message 0x001 has a worst-case response of 523.5 us (blocking 403 us + own frame 120.5 us), i.e. 9.6x margin to its 5 ms deadline; with one error frame in the window it is 706.0 us. On Classical CAN the same message would need 540.0 us on a lightly loaded bus, but the lower-priority messages grow to 5.7 ms because 21 frames compete; on FD the slowest message finishes in 1.65 ms. Improvement identified: the blocking term is dominated by the 64-byte diagnostic frame; limiting diagnostic frames to 32 B reduces B from 403 us to 243 us.

6.3 End-to-end latency budget (collision detection to brake command)

Table 6.3 - Worst-case latency budget

Step	Time (ms)
Radar/camera sample + fusion (Safety ECU)	4.00
Collision logic + E2E encode	1.00
CAN FD worst-case response (0x001), no error	0.52
Brake ECU Rx + E2E check + arbitration of request	1.00
Valve/pump driver activation	2.00
Total (requirement: <= 10 ms)	**8.52**

The budget leaves 1.48 ms (15 %) of margin. The bus contributes only 0.52 ms, so the delay is dominated by sensor fusion and actuation, not by the network.

6.4 Communication reliability and error handling

- Detection: bit monitoring, bit-stuffing, frame (form) check, CRC (HD = 6 for CAN-15 and for FD-17/21) and acknowledgement check. Any node detecting an error sends an error frame, so all nodes discard the frame and the sender retransmits automatically.
- Fault confinement: each node has a transmit error counter TEC (+8 per transmit error, -1 per success). TEC > 127 -> error-passive; TEC > 255 -> bus-off. Calculation: bus-off needs floor(255/8) + 1 = 32 consecutive transmit errors; recovery requires 128 x 11 recessive bits = 1408 bit times = 2.82 ms at 500 kbit/s. A failing node therefore removes itself from the bus quickly.

- Application-level protection (E2E): a 4-bit alive counter and CRC-8 in each safety message detect loss, repetition, insertion and masquerade that CAN's CRC cannot; two consecutive missing messages trigger the timeout reaction.

6.5 Processing requirements, power and memory

Brake ECU task set (WCET measured/assumed on a 200 MHz lockstep core). CPU utilisation U = Sum(C_i/T_i) = 62.5 %; the Liu-Layland rate-monotonic bound for n = 8 tasks is 72.4 %, and exact response-time analysis shows every task meets its period:

Table 6.4 - Brake ECU rate-monotonic schedulability

Task	Period (ms)	WCET (ms)	Response R (ms)	R <= T
WheelSpeedCapture	1.0	0.08	0.08	**YES**
CAN FD Rx/Tx handler	1.0	0.12	0.20	**YES**
ABS/EBD control loop	5.0	0.90	1.30	**YES**
EmergencyBrake manager	5.0	0.45	1.75	**YES**
Safety monitor/watchdog	5.0	0.20	1.95	**YES**
E2E check + signal mux	10.0	0.35	2.50	**YES**
Diagnostics (UDS)	20.0	0.80	3.50	**YES**
Self-test (RAM/Flash CRC slice)	50.0	2.00	7.85	**YES**

Utilisation is 62.5 % (< 70 % design limit); the exact analysis supersedes the pessimistic 72.4 % bound. Memory budgets in Table 1.4 carry >= 25 % margin: e.g. the Braking ECU's 256 KB RAM holds < 190 KB (stack, E2E buffers, CAN FD message RAM of 64 B x 32 buffers = 2 KB). Estimated electrical power: Engine 2.0 W, Braking 2.5 W, Safety 4.0 W, Body 1.2 W, Gateway 1.5 W, Infotainment 15 W - total about 26 W (2.2 A at 12 V); safety MCUs run at 0.5-1.5 W, which supports thermal design without active cooling; ECUs enter sleep mode (< 100 uA) after ignition-off via CAN partial networking.

6.6 Redundancy, fault tolerance and scalability

- Lockstep dual-core MCU + ECC + window watchdog (detects computation errors), redundant bus channel A' (single-fault tolerance of the communication path) and hydraulic fallback (mechanical safety net). If one bus channel has availability a = 0.999, two independent channels give 1-(1-a)^2 = 0.999999.
- **Scalability:** doubling or tripling the number of signals raises Bus A load only to 19.8 % and 29.7 % (Classical CAN: 54 % and 81 %). New ECUs can be added to an existing bus or to a new domain through the gateway. Beyond that, a 5 Mbit/s data phase, CAN XL or an Automotive-Ethernet backbone (for raw camera/radar data, which CAN FD cannot carry) is the migration path.
- **Suitability for safety-critical use:** ISO 26262 concepts (lockstep, E2E, safe states, freedom from interference) are in place; formal certification needs FMEDA, fault-injection and HIL evidence (open item, Section 7).

6.7 Failure scenarios and mitigation

Table 6.5 - Failure-mode analysis (six scenarios)

ID	Affected component	Effect on system	Mitigation	How reliable operation is maintained
F1	Braking ECU MCU (core / RAM / clock fault)	Wrong or no brake command; possible loss of ABS/EBR	Lockstep comparator + ECC + window watchdog detect fault in < 1 ms -> ECU enters safe state, de-energises valves, hydraulic (driver-pedal) braking remains; sends Fault flag on Bus A	Fault is detected before it propagates; Safety ECU sees missing heartbeat in 20 ms and warns driver; vehicle remains drivable with base braking.
F2	Bus A physical fault (open/short, transceiver failure, EMI burst)	Loss of safety messages (EBR, wheel speed) on primary channel	Redundant CAN FD channel A' carries safety-critical frames (0x001, 0x010, 0x080, 0x0C0); receivers accept first valid copy (alive counter de-duplicates); transceivers with bus-fault diagnosis	Single physical fault never removes both channels (separate harness routing); latency unchanged (same frame time).
F3	Infotainment ECU compromised / babbling (sends highest-priority frames or floods bus)	Starves Bus C; attempt to reach Bus A	Domain isolation: infotainment only on Bus C; gateway whitelists IDs and rate-limits (e.g. 0x4xx <= 1 frame / 10 ms); node drops to bus-off after 32 consecutive errors; bus guardian can cut power	Safety bus not electrically connected to the faulty node; worst-case response times on Bus A unchanged.
F4	Corrupted, lost or stale message (EMI burst, node reset)	Brake/Engine act on wrong data	CAN CRC + automatic retransmission; E2E counter + CRC-8 detects repetition, loss and masquerade; timeout (2 x period) -> use last valid value for <= 20 ms, then default safe value; one error frame adds 31 bit times + retransmission (+ 185 us)	R(0x001) rises 523.5 -> 706 us, still < 5 ms deadline (Section 6.3).
F5	Central gateway failure	Loss of cross-domain data (speed to HMI, diagnostics)	Safety domain never depends on gateway for control loops; Bus A messages continue; infotainment/body fall back to local defaults; warning lamp	Failure is confined to non-critical functions: graceful degradation.
F6	Radar/camera or wheel-speed sensor fault	False or missing collision data	Redundant sensing (radar + camera + wheel-speed cross-check), plausibility monitors; on disagreement system warns only (no autonomous braking)	Prevents false emergency braking; keeps driver-controlled braking.

6.8 Verification summary

Criterion	Evidence	Result
Real-time performance	All 10 Bus-A messages meet deadlines, even with one error frame; worst-case EBR response 523.5 us (0.52 ms) vs 5 ms; E2E latency 8.5 ms vs 10 ms budget	**PASS**
Communication reliability	CRC HD = 6, E2E protection, retransmission; redundant channel A'	**PASS**
CAN arbitration	Safety message always wins; max blocking 403 us (one 64 B frame) -> reduce diagnostic block to <= 32 B to cut to 243 us	**PASS**
Error handling	Five error types detected; bus-off after 32 errors; recovery 2.8 ms (128 x 11 recessive bits)	**PASS**
Fault tolerance / redundancy	Lockstep MCUs, dual bus, watchdog, safe states; single-fault tolerant for F1-F6	**PASS**
Scalability	Bus A load 9.9 % -> 19.8 % (2x), 29.7 % (3x); Classical CAN would reach 81 %	**PASS**
Processing requirements	Brake ECU utilisation 62.5 % (< 70 % limit), all tasks schedulable	**PASS**
Power & memory	Total ECU power ~ 27 W (~ 2.2 A at 12 V); sleep current < 100 uA per ECU; Flash/RAM budgets with >= 25 % margin	**PASS**
Safety suitability	ISO 26262 ASIL-D concepts (lockstep, E2E, safe state); certification evidence still to be produced (HIL, FMEDA)	**CONDITIONAL**

Final Engineering Decision

WOULD YOU DEPLOY THIS ARCHITECTURE? - YES

I would deploy this architecture for the first production release, subject to the pre-release conditions below. Every requirement of the System Requirement Statement is met with measured margin, and the remaining open items concern certification evidence, not architecture.

Table 7.1 - Defence of the decision

Basis	Justification
Compliance with requirements	All five parts of the Requirement Statement satisfied: deadlines (0.52 ms vs 5 ms; 8.5 ms vs 10 ms), ASIL-D mechanisms, domain isolation, bus load < 70 % with 3x growth, payloads up to 64 B.
Embedded architecture	Lockstep ASIL-D MCUs for control, SoC only where compute demands it; Brake ECU at 62.5 % load; no over-design.
Real-time communication	Priority arbitration guarantees the EBR frame wins; worst-case response analysis shows all deadlines met, also with one transmission error.
CAN / CAN FD suitability	Hybrid: FD where payload and speed are needed (load 27.0 % -> 9.9 %), Classical CAN where it is cheapest and sufficient.
Formal modelling	Mealy FSM + Petri net, executable and test-verified; shows safe-state behaviour and watchdog inhibition.
Fault tolerance	Six failure scenarios mitigated; no single fault removes braking; mechanical fallback.
Scalability	3x signal growth needs only 29.7 % of Bus A; gateway allows new domains; Ethernet path for ADAS data.
Implementation complexity	Moderate: one MCU family for 5 of 6 ECUs, standard AUTOSAR stack, three buses and one gateway; main extra complexity is the gateway and redundant channel, which are justified by the safety gain.

Conditions and residual risks before release

1. Run hardware-in-the-loop tests with fault injection (bus short, node babbling, power dips) to confirm the Section 6.3 latencies on real silicon.
2. Complete the ISO 26262 safety case: FMEDA of the Brake and Safety ECUs, PMHF < 10 FIT proof, HARA documentation.
3. Reduce the diagnostic/OTA frame on Bus A from 64 B to 32 B (or schedule updates only when the vehicle is parked) to cut the worst-case blocking from 403 us to 243 us.
4. Add message authentication (e.g. SecOC with truncated MAC) on the gateway for cyber-security.
5. Plan an Automotive-Ethernet backbone for camera/radar raw data in the next platform generation; CAN FD cannot carry it.

Had the answer been NO, the required redesign would have been the redundant channel A' and a lockstep upgrade of the Safety ECU; both are already part of this design, so no redesign is needed.

Appendix A - Python analysis code (can_analysis.py)

Run with: python can_analysis.py. It reproduces every number in Sections 4 and 6 (frame times, bus load, worst-case response times, CPU schedulability, decision matrix, bus-off and latency budget).

```
#!/usr/bin/env python3
"""
can_analysis.py - quantitative verification for the luxury-car networked embedded system
 - frame transmission times (Classical CAN, CAN FD)
 - bus load per network
 - worst-case response time (non-preemptive fixed-priority, Davis et al. 2007)
 - CPU utilisation (Rate-Monotonic) for the Brake ECU
 - weighted decision matrix for processor selection
 - bus-off calculation
Run:  python can_analysis.py   (writes results.json too)
"""
import math, json

# ---------------------------------------------------------------- frame models
def classical_bits(n, ext=False):
    """Worst-case bits of a Classical CAN data frame incl. stuffing + 3-bit IFS."""
    g = 54 if ext else 34
    return 8*n + g + 13 + math.floor((g + 8*n - 1)/4)

def classical_time_us(n, bitrate):
    return classical_bits(n) * 1e6 / bitrate

def fd_time_us(n, nominal, data):
    """CAN FD (11-bit ID, BRS=1) worst-case frame time in microseconds.
    n must be a legal FD payload length (0..8,12,16,20,24,32,48,64)."""
    # arbitration phase (nominal rate): SOF+ID+RRS+IDE+FDF+res = 16 bits, BRS = 1
    arb_dyn = 16
    nom_bits = 16 + 1 + math.floor((arb_dyn - 1)/4)         # + stuff bits
    nom_bits += 1 + 1 + 7 + 3                               # ACK, ACKdel, EOF, IFS
    crc_len = 17 if n <= 16 else 21
    crc_field = crc_len + (5 if n <= 16 else 6)             # fixed stuff bits
    dyn = 1 + 4 + 8*n                                       # ESI + DLC + data
    data_bits = dyn + math.floor((dyn - 1)/4) + 4 + crc_field + 1   # + SBC + CRC + CRC delim
    return nom_bits*1e6/nominal + data_bits*1e6/data

def legal_fd(n):
    for L in (0,1,2,3,4,5,6,7,8,12,16,20,24,32,48,64):
        if n <= L: return L

# ---------------------------------------------------------------- message set
# (can_id, name, sender, period_ms, payload_bytes, deadline_ms)
BUS_A = [
 (0x001,"EmergencyBrakeRequest","Safety ECU",10, 8, 5),
 (0x010,"SafetyHeartbeat",      "Safety ECU",10, 8, 10),
 (0x080,"WheelSpeed",           "Brake ECU", 10, 8, 10),
 (0x0C0,"BrakeStatus",          "Brake ECU", 10, 8, 10),
 (0x100,"EngineTorque",         "Engine ECU",10, 8, 10),
 (0x110,"TorqueLimitReq",       "Brake ECU", 10, 8, 10),
 (0x120,"EngineData",           "Engine ECU",20,16, 20),
 (0x200,"RadarObjectList",      "Safety ECU",20,32, 20),
```

```
 (0x300,"GatewayRouted",        "Gateway",   50, 8, 50),
 (0x7E0,"DiagSoftwareBlock",    "Gateway",  100,64,100),
]
BUS_B = [  # classical CAN 125 kbit/s
 (0x100,"DoorLockStatus","BCM",50,8,50),(0x110,"LightingCmd","BCM",20,8,20),
 (0x120,"WiperStatus","BCM",100,8,100),(0x130,"WindowStatus","BCM",100,8,100),
 (0x140,"ClimateData","BCM",100,8,100),(0x150,"ImmobiliserAuth","BCM",100,8,100),
 (0x160,"BodyGatewayInfo","Gateway",50,8,50),
]
BUS_C = [  # CAN FD infotainment 500k/2M

(0x400,"MediaControl","Infotainment",20,16,20),(0x410,"NavigationData","Infotainment",50,64,
50),

(0x420,"TelematicsPacket","Infotainment",50,64,50),(0x430,"DisplayCluster","Infotainment",20,
32,20),
 (0x440,"VehicleInfoToHMI","Gateway",20,32,20),
]

def expand_classical(msgs):
    """split payloads > 8 B into several 8-byte frames (priority-adjacent IDs)."""
    out=[]
    for (i,nm,s,T,n,D) in msgs:
        k=math.ceil(n/8)
        for j in range(k):
            out.append((i+j, nm+("" if k==1 else f"[{j}]"), s, T, 8 if j<k-1 else n-8*(k-1) or 8, D))
    return sorted(out)

def frames(msgs, mode, nom, data=None):
    res=[]
    if mode=="classical": msgs=expand_classical(msgs)
    for (i,nm,s,T,n,D) in msgs:
        C = classical_time_us(n,nom) if mode=="classical" else fd_time_us(legal_fd(n),nom,data)
        res.append(dict(id=i,name=nm,sender=s,T=T*1000.0,n=n,D=D*1000.0,C=C))
    return res

def bus_load(fr): return sum(f["C"]/f["T"] for f in fr)

# ---------------------------------------------------------------- WCRT analysis
def wcrt(fr, bitrate_nom, errors=0):
    """Non-preemptive FP response time. errors = number of error frames in busy window."""
    fr = sorted(fr, key=lambda f:f["id"])
    tau = 1e6/bitrate_nom
    out=[]
    Cmax = max(f["C"] for f in fr)
    for k,f in enumerate(fr):
        hp = fr[:k]; lp = fr[k+1:]
        B = max([x["C"] for x in lp], default=0)
        w = B
        for _ in range(1000):
            E = errors*(31*tau + max([x["C"] for x in hp]+[f["C"]]))
            wn = B + E + sum(math.ceil((w+tau)/x["T"])*x["C"] for x in hp)
            if abs(wn-w)<1e-9: break
            w = wn
        R = w + f["C"]
        out.append(dict(id=f["id"],name=f["name"],C=f["C"],B=B,R=R,D=f["D"],ok=R<=f["D"]))
    return out
```

```
# ---------------------------------------------------------------- CPU utilisation
BRAKE_TASKS = [  # name, period ms, WCET ms
 ("WheelSpeedCapture",1.0,0.08),("ABS/EBD control loop",5.0,0.90),
 ("EmergencyBrake manager",5.0,0.45),("Safety monitor/watchdog",5.0,0.20),
 ("CAN FD Rx/Tx handler",1.0,0.12),("E2E check + signal mux",10.0,0.35),
 ("Diagnostics (UDS)",20.0,0.80),("Self-test (RAM/Flash CRC slice)",50.0,2.00)]
def cpu_util(tasks):
    U=sum(c/t for _,t,c in tasks); n=len(tasks)
    return U, n*(2**(1/n)-1)
def rta_cpu(tasks):
    tasks=sorted(tasks,key=lambda x:x[1]); res=[]
    for i,(nm,T,C) in enumerate(tasks):
        R=C
        while True:
            Rn=C+sum(math.ceil(R/t)*c for _,t,c in tasks[:i])
            if abs(Rn-R)<1e-12: break
            R=Rn
            if R>T*10: break
        res.append((nm,T,C,R,R<=T))
    return res

# ---------------------------------------------------------------- decision matrix
CRIT=[("Processing capability",15),("Memory",10),("Power",10),("Real-time response",20),
      ("Integration",10),("Cost/complexity",15),("Automotive suitability",20)]
SCORES_CTRL={ # 1-5 : control ECUs (engine, brake, safety, body)
 "MCU":[3,3,5,5,4,5,5],"MPU":[4,5,2,2,3,3,2],"DSP":[4,3,3,4,2,3,3],"SoC":[5,5,2,3,5,2,3]}
SCORES_INFO={ # infotainment
 "MCU":[1,1,5,4,2,5,4],"MPU":[4,4,3,2,3,3,3],"DSP":[3,2,3,3,2,3,2],"SoC":[5,5,3,3,5,3,4]}
def wsum(sc): return sum(s*w for s,(_,w) in zip(sc,CRIT))/100.0

# ---------------------------------------------------------------- main
if __name__=="__main__":
    R={}
    print("== Frame times (us) ==")
    ft={}
    for n in (8,16,32,64):
        c500=classical_time_us(min(n,8),500e3)
        ft[n]=dict(fd=fd_time_us(n,500e3,2e6), fd5=fd_time_us(n,500e3,5e6),
                   classical_frames=math.ceil(n/8),
                   classical_total=math.ceil(n/8)*classical_time_us(8,500e3))
        print(n, ft[n])
    R["frame_times"]=ft
    R["classical8_bits"]=classical_bits(8); R["classical8_us"]=classical_time_us(8,500e3)

    A_cl = frames(BUS_A,"classical",500e3)
    A_fd = frames(BUS_A,"fd",500e3,2e6)
    B_cl = frames(BUS_B,"classical",125e3)
    C_fd = frames(BUS_C,"fd",500e3,2e6)
    C_cl = frames(BUS_C,"classical",500e3)

R["load"]=dict(A_classical=bus_load(A_cl),A_fd=bus_load(A_fd),B_classical=bus_load(B_cl),
               C_fd=bus_load(C_fd),C_classical=bus_load(C_cl),
               A_cl_frames=len(A_cl),A_fd_frames=len(A_fd))
    print("== Bus load =="); print(json.dumps(R["load"],indent=1))
    R["A_fd_frames"]=A_fd
    R["wcrt_A_fd"]=wcrt(A_fd,500e3,0); R["wcrt_A_fd_err"]=wcrt(A_fd,500e3,1)
```

```
    R["wcrt_A_cl"]=wcrt(A_cl,500e3,0); R["wcrt_A_cl_err"]=wcrt(A_cl,500e3,1)
    for key in ("wcrt_A_fd","wcrt_A_fd_err","wcrt_A_cl"):
        print("==",key)
        for r in R[key]: print(f"  0x{r['id']:03X} {r['name']:22s} C={r['C']:7.1f} B={r['B']:7.1f}
R={r['R']:8.1f} D={r['D']:8.0f} {'OK' if r['ok'] else 'MISS'}")
    # scalability: scale all periodic message rates x2 (double signal count)
    sc=[dict(f,T=f["T"]/2) for f in A_fd]
    R["load_A_fd_x2"]=bus_load(sc)
    sc3=[dict(f,T=f["T"]/3) for f in A_fd]
    R["load_A_fd_x3"]=bus_load(sc3)
    # classical on bus A with doubling
    R["load_A_cl_x2"]=bus_load([dict(f,T=f["T"]/2) for f in A_cl])
    print("Load A FD x2 / x3 / classical x2:",
R["load_A_fd_x2"],R["load_A_fd_x3"],R["load_A_cl_x2"])

    U,bound=cpu_util(BRAKE_TASKS);
R["cpu"]=dict(U=U,bound=bound,rta=rta_cpu(BRAKE_TASKS))
    print("== Brake ECU CPU: U=%.3f  RM bound=%.3f"%(U,bound))
    for r in R["cpu"]["rta"]: print("  ",r)

    R["matrix_ctrl"]={k:wsum(v) for k,v in SCORES_CTRL.items()}
    R["matrix_info"]={k:wsum(v) for k,v in SCORES_INFO.items()}
    print("Decision matrix control:",R["matrix_ctrl"]); print("Infotainment:",R["matrix_info"])

    # bus-off: TEC +8 per transmit error, bus-off when TEC > 255
    errs=math.floor(255/8)+1; R["busoff_errors"]=errs
    # recovery: 128 occurrences of 11 recessive bits = 1408 bit times at 500 kbit/s
    R["busoff_recovery_ms"]=128*11/500e3*1e3
    print("bus-off after",errs,"consecutive tx errors; recovery",R["busoff_recovery_ms"],"ms")

    # end-to-end latency budget (collision detect -> brake actuator command), worst case
    ebr = next(r for r in R["wcrt_A_fd"] if r["id"]==0x001)
    budget=[("Radar/camera sample + fusion (Safety ECU)",4.0),("Collision logic + E2E
encode",1.0),
            ("CAN FD worst-case response (0x001), no error",ebr["R"]/1000),
            ("Brake ECU Rx + E2E check + arbitration of request",1.0),("Valve/pump driver
activation",2.0)]
    R["latency"]=budget; R["latency_total"]=sum(b for _,b in budget)
    print("E2E latency budget:",R["latency_total"],"ms (limit 10 ms)")
    json.dump(R,open("results.json","w"),indent=1,default=str)
```

Appendix B - C implementation of the emergency-braking FSM (aeb_fsm.c)

Compile and run: gcc -Wall -o aeb_fsm aeb_fsm.c && ./aeb_fsm . One call of aeb_step() is one 5 ms control tick; outputs are produced on transitions (Mealy).

```
/* aeb_fsm.c - Emergency-braking supervisor (Mealy FSM) for the Safety ECU
 * Build/test:  gcc -Wall -o aeb_fsm aeb_fsm.c && ./aeb_fsm
 * One call of aeb_step() = one 5 ms control tick. Outputs are produced on transitions.
 */
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>

typedef enum { S_INIT, S_STANDBY, S_MONITOR, S_WARNING, S_PREBRAKE,
               S_EMERGENCY, S_HOLD, S_DEGRADED, S_SAFE } state_t;

typedef struct {            /* inputs / events sampled each tick */
    bool  self_test_ok;     /* power-on self-test result           */
    bool  fault;            /* lockstep / E2E / sensor fault       */
    bool  ignition_on;
    float ttc_s;            /* time-to-collision from sensor fusion */
    float speed_kmh;
    bool  driver_brake;     /* driver already braking hard         */
    bool  driver_override;  /* steering/accelerator override       */
    bool  bus_alive;        /* heartbeat of Brake ECU received     */
} inputs_t;

typedef struct {            /* outputs / actions                   */
    bool  warn_hmi;         /* buzzer + cluster warning            */
    bool  prefill;          /* hydraulic pre-fill                  */
    float decel_req_ms2;    /* deceleration request on CAN id 0x001 */
    bool  limit_torque;     /* ask Engine ECU to cut torque        */
} outputs_t;

#define TTC_WARN   2.6f
#define TTC_PRE    1.8f
#define TTC_BRAKE  1.0f
#define MIN_SPEED  8.0f
#define HOLD_TICKS 200      /* 200 x 5 ms = 1 s hold after stop    */

static state_t st = S_INIT;
static int hold_cnt = 0;

outputs_t aeb_step(const inputs_t *in)
{
    outputs_t o = {false, false, 0.0f, false};

    if (st != S_INIT && st != S_SAFE && (in->fault || !in->bus_alive)) {
        st = (in->fault) ? S_SAFE : S_DEGRADED;   /* fail-safe / degraded */
        return o;
    }
    switch (st) {
    case S_INIT:
        if (in->self_test_ok) st = S_STANDBY; else st = S_SAFE;
        break;
    case S_STANDBY:
```

```
        if (in->ignition_on && in->speed_kmh > MIN_SPEED) st = S_MONITOR;
        break;
    case S_MONITOR:
        if (in->speed_kmh <= MIN_SPEED) st = S_STANDBY;
        else if (in->ttc_s < TTC_WARN) { st = S_WARNING; o.warn_hmi = true; }
        break;
    case S_WARNING:
        o.warn_hmi = true;
        if (in->driver_override || in->ttc_s >= TTC_WARN + 0.3f) st = S_MONITOR;
        else if (in->ttc_s < TTC_PRE) { st = S_PREBRAKE; o.prefill = true; o.decel_req_ms2 =
2.0f; }
        break;
    case S_PREBRAKE:
        o.warn_hmi = o.prefill = true; o.decel_req_ms2 = 2.0f;
        if (in->driver_override) st = S_MONITOR;
        else if (in->ttc_s < TTC_BRAKE && !in->driver_brake) {
            st = S_EMERGENCY; o.decel_req_ms2 = 9.0f; o.limit_torque = true; }
        break;
    case S_EMERGENCY:
        o.warn_hmi = o.prefill = o.limit_torque = true; o.decel_req_ms2 = 9.0f;
        if (in->speed_kmh < 1.0f) { st = S_HOLD; hold_cnt = 0; }
        else if (in->driver_override) st = S_MONITOR;
        break;
    case S_HOLD:
        o.decel_req_ms2 = 3.0f;                 /* keep vehicle stationary */
        if (++hold_cnt >= HOLD_TICKS) st = S_STANDBY;
        break;
    case S_DEGRADED:
        o.warn_hmi = true;                      /* driver-only braking     */
        if (in->bus_alive && !in->fault) st = S_STANDBY;
        break;
    case S_SAFE:
        o.warn_hmi = true;                      /* latched until restart   */
        break;
    }
    return o;
}

/* ------------------------------ self-checking test bench ------------------ */
static const char *names[] = {"INIT","STANDBY","MONITOR","WARNING","PREBRAKE",
                              "EMERGENCY","HOLD","DEGRADED","SAFE"};
int main(void)
{
    inputs_t in = {true,false,true,9.9f,50.0f,false,false,true};
    float ttc_seq[] = {9.9f,9.9f,2.5f,2.0f,1.7f,0.9f,0.8f,0.7f};
    int tick = 0, pass = 1;
    printf("tick  TTC   state      warn pre  decel torque\n");
    aeb_step(&in); in.ttc_s = 9.9f;             /* INIT -> STANDBY   */
    for (int i = 0; i < 8; i++, tick++) {
        in.ttc_s = ttc_seq[i];
        if (i >= 5) in.speed_kmh -= 20.0f;          /* braking slows car */
        outputs_t o = aeb_step(&in);
        printf("%3d  %4.1f  %-9s   %d    %d   %4.1f   %d\n", tick, in.ttc_s,
               names[st], o.warn_hmi, o.prefill, o.decel_req_ms2, o.limit_torque);
    }
    pass &= (st == S_EMERGENCY || st == S_HOLD);
    in.fault = true; aeb_step(&in);             /* inject fault      */
```

```
    printf("fault injected -> %s\n", names[st]);
    pass &= (st == S_SAFE);
    printf("TEST %s\n", pass ? "PASSED" : "FAILED");
    return !pass;
}
```

Appendix C - Assumptions and references

Assumptions

- Message periods, payloads and deadlines (Table 3.2), WCETs (Table 6.4), memory/power budgets and DMIPS figures are design estimates typical for the ECU class; they should be replaced by measured values when a target MCU is chosen.
- Frame times use worst-case bit stuffing and the frame-format of ISO 11898-1:2015 for 11-bit identifiers. The CAN FD figure is an engineering estimate (dynamic stuffing bounded by floor((n-1)/4)).
- Error model: at most one error frame (31 bit times) plus one retransmission in any busy window; sporadic emergency message minimum inter-arrival 10 ms.
- Decision-matrix weights and scores are the author's judgement; the ranking is robust (MCU leads for control ECUs by about 1 point).